Advanced DBMS SQL Queries

On Industry-Focussed Use Cases

Dr. Sumaiya Tabassum Nimi
Assistant Professor, Dept. of ECE — North South University

Introduction

This handout provides a set of **intermediate-level SQL problems** that focus on four important topics:

- **JOIN** – combining data from multiple tables.
- **GROUP BY** – aggregating rows into groups.
- **HAVING** – filtering groups based on aggregate conditions.
- **Window functions with PARTITION BY** – performing row-wise analytics (e.g., ranking, running totals) without collapsing rows.

The examples are based on realistic scenarios inspired by companies like **Netflix**, **Amazon**, and **Uber**. For each scenario, we first describe the schema, then provide practice queries and detailed solutions.

Schema 1: Streaming Platform (e.g., Netflix / Amazon Prime)

Consider a video streaming platform with the following simplified schema:

Tables

Users

```
Users(
    user_id      INT PRIMARY KEY,
    name         VARCHAR(100),
    country      VARCHAR(50)
);
```

Movies

```
Movies(
    movie_id     INT PRIMARY KEY,
    title        VARCHAR(200),
    genre        VARCHAR(50)
);
```

WatchHistory

```
WatchHistory(
    user_id      INT,
    movie_id     INT,
    watch_date   DATE,
    watch_minutes INT,
    PRIMARY KEY(user_id, movie_id, watch_date),
    FOREIGN KEY(user_id) REFERENCES Users(user_id),
    FOREIGN KEY(movie_id) REFERENCES Movies(movie_id)
);
```

Problem 1 (JOIN + GROUP BY + HAVING)

Task: Find all **genres** where there are at least **5 distinct users** who have watched movies from that genre.

For each such genre, output:

- genre
- user_count (number of distinct users who watched that genre)

Solution:

```
SELECT
    m.genre,
    COUNT(DISTINCT wh.user_id) AS user_count
FROM WatchHistory AS wh
JOIN Movies AS m
    ON wh.movie_id = m.movie_id
GROUP BY m.genre
HAVING COUNT(DISTINCT wh.user_id) >= 5;
```

Explanation:

- We **JOIN** `WatchHistory` with `Movies` to know the **genre** of each watched movie.
- `GROUP BY m.genre` aggregates all watches into genre-based groups.
- `COUNT(DISTINCT wh.user_id)` counts how many unique users watched that genre.
- `HAVING COUNT(DISTINCT wh.user_id) >= 5` filters to only popular genres with at least 5 distinct viewers.

Problem 2 (Window Function with PARTITION BY)

Task: For each **genre**, list the **top 3 most-watched movies** (by total watch minutes).

Output:

- genre
- title
- total_minutes
- rank_in_genre (1 = highest total minutes)

Solution:

```
WITH MovieTotals AS (
    SELECT
        m.genre,
        m.title,
        SUM(wh.watch_minutes) AS total_minutes
    FROM WatchHistory AS wh
    JOIN Movies AS m
        ON wh.movie_id = m.movie_id
    GROUP BY m.genre, m.title
)
SELECT
    genre,
    title,
    total_minutes,
    ROW_NUMBER() OVER(
        PARTITION BY genre
        ORDER BY total_minutes DESC
    ) AS rank_in_genre
FROM MovieTotals
WHERE ROW_NUMBER() OVER(
        PARTITION BY genre
        ORDER BY total_minutes DESC
    ) <= 3;
```

Note: In many SQL dialects (e.g., PostgreSQL), you cannot repeat the same window expression in `WHERE`. Instead, wrap it again or use another CTE:

A clearer, dialect-friendly version:

```
WITH MovieTotals AS (
    SELECT
        m.genre,
        m.title,
        SUM(wh.watch_minutes) AS total_minutes
    FROM WatchHistory AS wh
    JOIN Movies AS m
        ON wh.movie_id = m.movie_id
    GROUP BY m.genre, m.title
),
RankedMovies AS (
    SELECT
```

```
        genre,
        title,
        total_minutes,
        ROW_NUMBER() OVER(
            PARTITION BY genre
            ORDER BY total_minutes DESC
        ) AS rank_in_genre
    FROM MovieTotals
)
SELECT *
FROM RankedMovies
WHERE rank_in_genre <= 3;
```

Explanation:

- First, aggregate per movie using `GROUP BY genre, title`.
- Then, in `RankedMovies`, we use **ROW_NUMBER() OVER(PARTITION BY genre ORDER BY total_minutes DESC)** to rank movies **within each genre**.
- Finally, we filter to only the top 3 rows per genre using `WHERE rank_in_genre <= 3`.

Schema 2: E-Commerce Platform (e.g., Amazon)

Tables

Customers

```
Customers(
    customer_id   INT PRIMARY KEY,
    name          VARCHAR(100),
    country       VARCHAR(50)
);
```

Orders

```
Orders(
    order_id      INT PRIMARY KEY,
    customer_id   INT,
    order_date    DATE,
    total_amount  DECIMAL(10,2),
    FOREIGN KEY(customer_id) REFERENCES Customers(customer_id)
);
```

OrderItems

```
OrderItems(
    order_id      INT,
```

```
    product_id    INT,
    quantity      INT,
    line_amount   DECIMAL(10,2),
    PRIMARY KEY(order_id, product_id),
    FOREIGN KEY(order_id) REFERENCES Orders(order_id)
);
```

Problem 3 (JOIN + GROUP BY)

Task: For each **customer**, find the **total number of orders** and the **total amount spent**. Only include customers who have placed at least **3 orders**.

Output:

- customer_id
- name
- order_count
- total_spent

Solution:

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM Customers AS c
JOIN Orders AS o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(o.order_id) >= 3;
```

Explanation:

- **JOIN** connects customers to their orders.
- `GROUP BY c.customer_id, c.name` aggregates per customer.
- `COUNT(o.order_id)` gives how many orders; `SUM(o.total_amount)` gives how much they spent.
- `HAVING COUNT(o.order_id) >= 3` keeps only frequent customers.

Schema 3: Ride-Sharing Platform (e.g., Uber)

Tables

Riders

```
Riders(
    rider_id     INT PRIMARY KEY,
    name         VARCHAR(100),
    city         VARCHAR(50)
);
```

Trips

```
Trips(
    trip_id      INT PRIMARY KEY,
    rider_id     INT,
    trip_date    DATE,
    distance_km  DECIMAL(6,2),
    fare_amount  DECIMAL(8,2),
    FOREIGN KEY(rider_id) REFERENCES Riders(rider_id)
);
```

Problem 5 (GROUP BY + HAVING)

Task: Find all cities where the **average trip fare** is more than **15 USD**.

Output:

- city
- avg_fare

Solution:

```
SELECT
    r.city,
    AVG(t.fare_amount) AS avg_fare
FROM Trips AS t
JOIN Riders AS r
    ON t.rider_id = r.rider_id
GROUP BY r.city
HAVING AVG(t.fare_amount) > 15;
```

Explanation:

- **JOIN** Riders with Trips to know the city for each trip.
- `GROUP BY r.city` aggregates trips by city.
- `HAVING AVG(t.fare_amount) > 15` filters to high-fare cities.

Problem 6 (Window Function: Ranking Within City)

Task: Within each city, rank riders by their **total fare amount** (highest first).

Show only the top 2 riders per city.

Output:

- city
- rider_id
- total_fare
- rank_in_city

Solution:

```
WITH RiderTotals AS (
    SELECT
        r.city,
        r.rider_id,
        SUM(t.fare_amount) AS total_fare
    FROM Trips AS t
    JOIN Riders AS r
        ON t.rider_id = r.rider_id
    GROUP BY r.city, r.rider_id
),
RankedRiders AS (
    SELECT
        city,
        rider_id,
        total_fare,
        RANK() OVER(
            PARTITION BY city
            ORDER BY total_fare DESC
        ) AS rank_in_city
    FROM RiderTotals
)
SELECT *
FROM RankedRiders
WHERE rank_in_city <= 2
ORDER BY city, rank_in_city;
```

Explanation:

- First, aggregate by `city, rider_id` to get each rider's total fare per city.
- Then, use `RANK() OVER(PARTITION BY city ORDER BY total_fare DESC)` to rank riders within their city.
- Finally, filter to the top 2 riders per city using `WHERE rank_in_city <= 2`.